CS 316: Introduction to Deep Learning

Linear Regression
Week 5

Dr Abdul Samad

Lecture Outline

- Linear Regression Model
- Single Layer Neural Network
- Evaluating Linear Regression Model
- Training Linear Regression Model
- Optimization
  - Gradient Descent
  - Learning Rate
  - Mini Batch Stochastic Gradient Descent

House Price Prediction

- Predict price of the house based on its features.
- The features in consideration are:
  - Total Area
  - Number of Bedrooms
  - Number of Bathrooms

Figure taken from medium

House Price Prediction

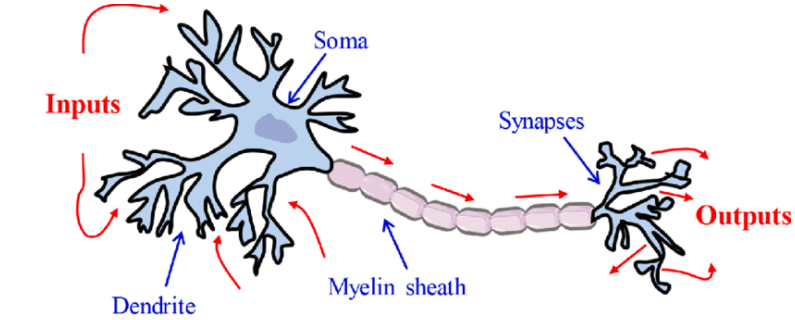
The simple linear model makes the following assumptions:

- Area, number of bedrooms, and number of bathrooms are key factors influencing house price and are denoted by x_1, x_2, x_3 .
- The sale price $\hat{y}$ is the weighted sum of the key factors and the bias.

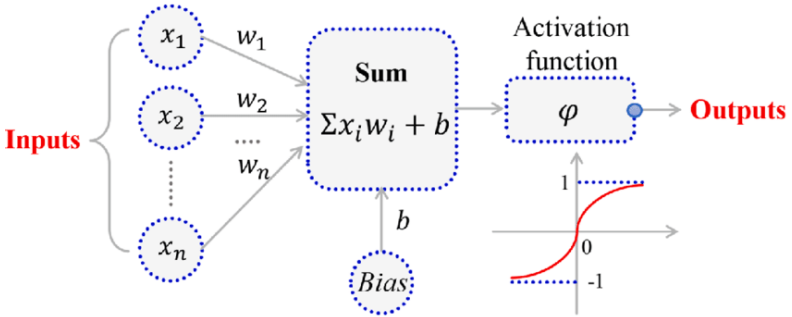
$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

- w_1, w_2, w_3 represent the weights and b represents the bias term.

Biological Neuron vs Artificial Neuron



(a) Biological neuron



(b) Artificial neuron

Figure taken from ResearchGate

Linear Model

- Given n dimensional input $\boldsymbol{x} = [x_1\ x_2 \cdots x_n]^T$, the linear model has n weights $\boldsymbol{w} = [w_1, w_2, \cdots, w_n]^T$ and the bias b, $\hat{y}$ is the weighted sum of the inputs and the bias.

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

- In vectorized notation, the output $\hat{y}$ can be expressed as follows:

$$\hat{y} = \langle \boldsymbol{w}, \boldsymbol{x} \rangle + b$$

Single Layer Neural Network

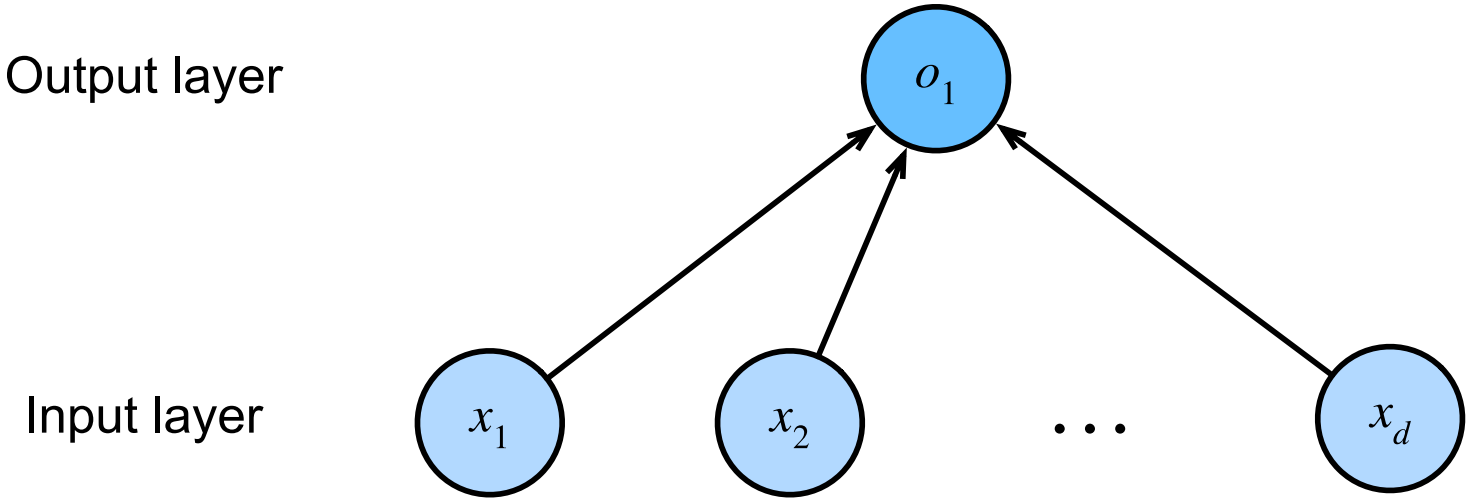


Figure taken from [d2l.ai]

Evaluating Accuracy of the Model

- Compare the actual and predicted values to evaluate the accuracy of the model
- For example, comparing the actual price of the house to the expected price of a house.
- Give the truth value y and predicted value $\hat{y}$, we can evaluate the accuracy of the model by computing square loss which is defined as follows:

$$l(y, \hat{y}) = (y - \hat{y})^2$$

Training Data

- $\boldsymbol{X} = [\boldsymbol{x}_1\ \boldsymbol{x}_2\ \cdots \boldsymbol{x}_n]^T$ where $\boldsymbol{x}_i$ is the i^{th} input vector i.e. features of the i^{th} house.
- x_i^j is the jth feature of the i^{th} input vector i.e. Total Area of the i^{th} house.
- $\boldsymbol{y} = [y_1\ y_2 \cdots y_n]$ where y_i is the ith output value i.e. house price of the i^{th} house.

Training Loss of a Linear Regression Model

$$l(\boldsymbol{w}, b) = \frac{1}{2}(\boldsymbol{w}^T \boldsymbol{x}_i + b - y_i)^2$$

$$l(\boldsymbol{w}, b) = \frac{1}{2}(\hat{y}_i - y_i)^2$$

$$L(\boldsymbol{w}, b) = \frac{1}{n}\sum_{i=1}^{n} l(\boldsymbol{w}, b)$$

$$L(\boldsymbol{w}, b) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{2}(\hat{y}_i - y_i)^2$$

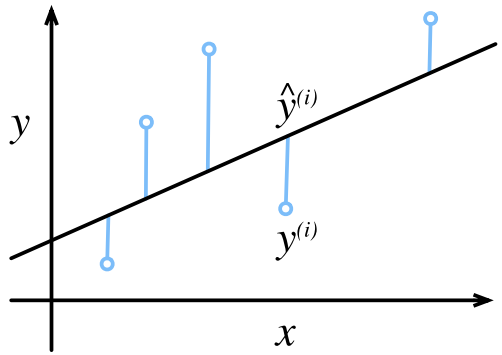


Figure taken from d2l.ai

Vectorized Training Loss

- Concatenate a column of ones to the input and bias into the weights.

$$\boldsymbol{X} = [\,1 \;\; \boldsymbol{X}], \boldsymbol{w} = \begin{bmatrix} b \\ \boldsymbol{w} \end{bmatrix}$$

- Vectorized Loss function is defined as

$$L(\boldsymbol{w}) = \frac{1}{n} \|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|_2^2$$

- Minimize the loss

$$\boldsymbol{w}^* = argmin_w L(\boldsymbol{w})$$

Closed Form Solution

Compute $\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}}$ using chain rule and then set it equal to 0 to find $\boldsymbol{w}^*$

$$\boldsymbol{a} = \boldsymbol{X}\boldsymbol{w}, \boldsymbol{b} = \boldsymbol{a} - \boldsymbol{y}, z = \|\boldsymbol{b}\|_2^2$$

$$\frac{\partial z}{\partial \boldsymbol{b}} = 2\boldsymbol{b}^{\boldsymbol{T}}, \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}} = \boldsymbol{I}, \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{w}} = \boldsymbol{X}$$

$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}} = \frac{\partial z}{\partial \boldsymbol{b}} \cdot \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}} \cdot \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{w}}$$

$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}} = 2(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^T \boldsymbol{X}$$

$$2(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^T \boldsymbol{X} = 0$$

$$\boldsymbol{w}^T \boldsymbol{X}^T \boldsymbol{X} - \boldsymbol{y}^T \boldsymbol{X} = \boldsymbol{0}$$

$$\boldsymbol{w}^T \boldsymbol{X}^T \boldsymbol{X} = \boldsymbol{y}^T \boldsymbol{X}$$

$$\boldsymbol{X}^T \boldsymbol{X}\boldsymbol{w} = \boldsymbol{X}^T \boldsymbol{y}$$

$$\boldsymbol{w}^* = (\boldsymbol{X}^T \boldsymbol{X})^{-1} \boldsymbol{X}^T \boldsymbol{y}$$

Gradient Descent Algorithm

- Choose initial values for the weights $\boldsymbol{w}_0$
- Update weights for $t = 1, 2, 3, \cdots$ using the following equation:

$$\boldsymbol{w}_t = \boldsymbol{w}_{t-1} - \alpha \frac{\partial l(\boldsymbol{w})}{\partial \boldsymbol{w}}$$

- α is a hyperparameter called learning that specifies step length.

1D Gradient Descent Visualization

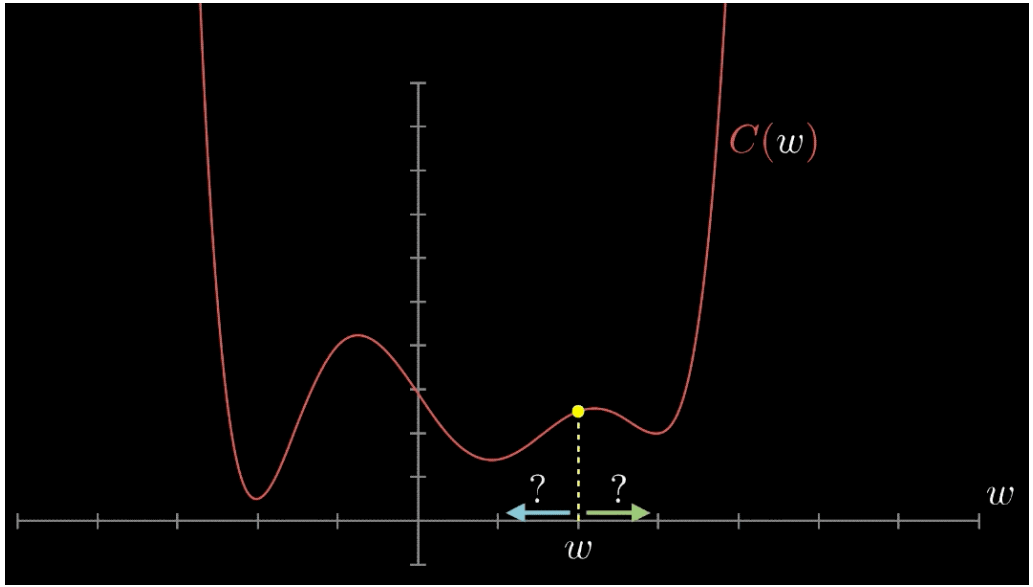


Figure taken from medium

2D Gradient Descent Visualization

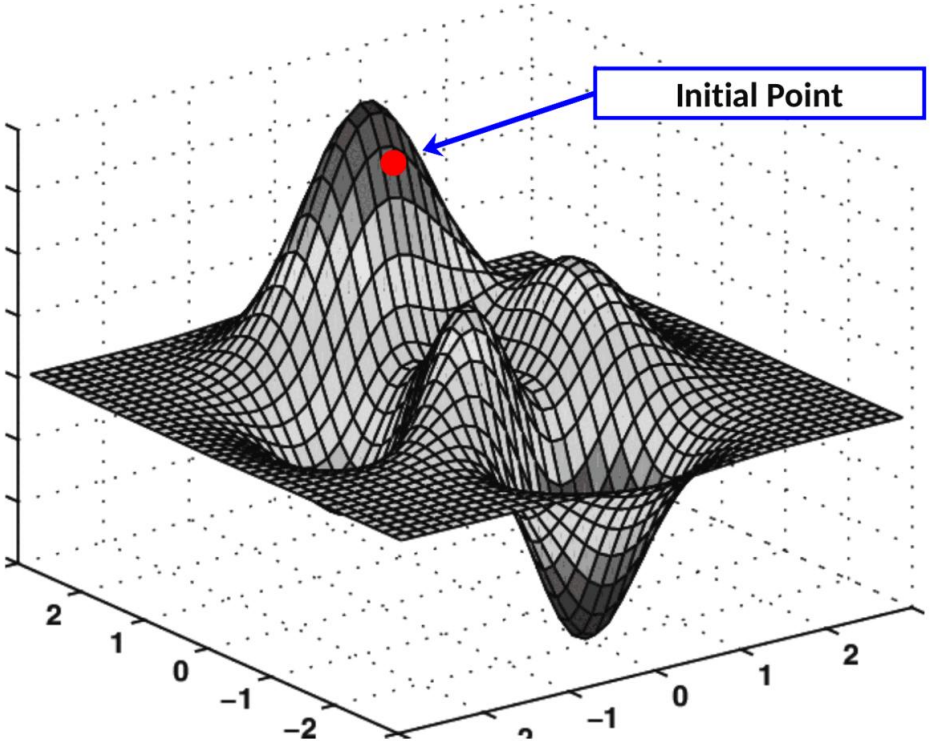


Figure taken from Towards AI

Learning Rate

- When the learning rate is too small, the loss function converges is slowly.
- When the learning is too big, the loss function may overshoot and even diverge.

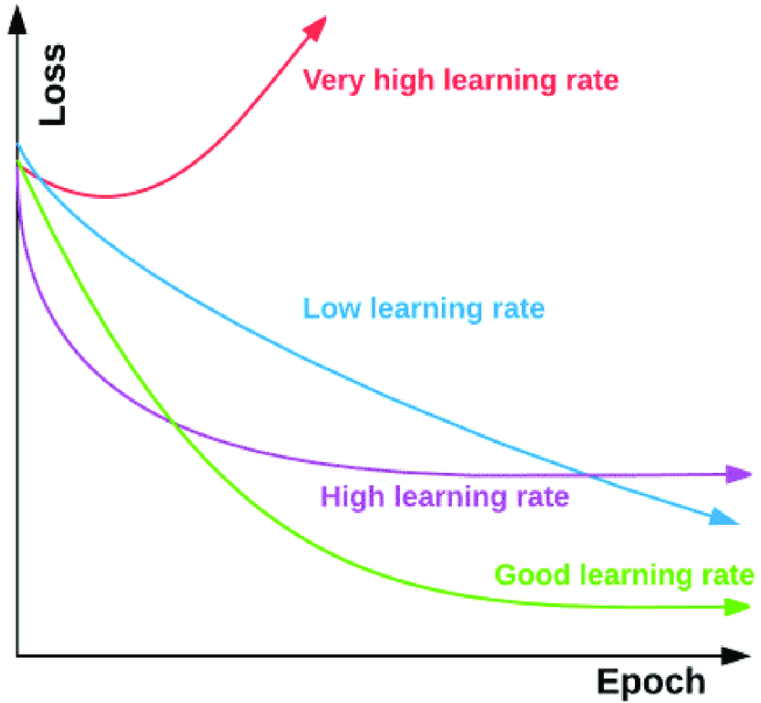


Figure taken from ResearchGate

Mini-Batch Stochastic Gradient Descent

- If the number of samples is large, a single iteration over the entire data set is not a viable technique.
- We can choose a fixed number of instances from set β at random, call it a batch, and perform one step of gradient descent, known as mini-batch stochastic gradient descent.
- An extreme case would be to choose one sample at random and then perform a gradient descent step.

Mini Batch Stochastic Gradient Descent

- Randomly initialize the model parameters, $\boldsymbol{w}$ and b
- Iteratively sample random mini batch β from the data.
- Update the parameters in the direction of the negative gradient.

$$\boldsymbol{w} = \boldsymbol{w} - \frac{\eta}{|\beta|} \sum_{i \in \beta_t} \partial_{\boldsymbol{w}} l^{(i)}(\boldsymbol{w}, b)$$

$$\boldsymbol{w} = \boldsymbol{w} - \frac{\eta}{|\beta|} \sum_{i \in \beta_t} \boldsymbol{x}^{(i)} (\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b - y^{(i)})$$

$$b = b - \frac{\eta}{|\beta|} \sum_{i \in \beta_t} \partial_b l^{(i)}(\boldsymbol{w}, b)$$

$$b = b - \frac{\eta}{|\beta|} \sum_{i \in \beta_t} (\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b - y^{(i)})$$

Batch Size

- Not too small
  - Workload is low, making it hard to fully utilize computational power.
- Not too big
  - Memory is not sufficient. Wasted computation if all x_i are identical

References

- https://d2l.ai/chapter_linear-regression/index.html

CS 316: Introduction to Deep Learning

SoftMax Regression
Week 6

Dr Abdul Samad

Lecture Outline

- Regression vs Classification
- Multiclass Classification
- Network Architecture
- One-hot Encoding
- Softmax
- Cross entropy loss

Regression vs Classification

- Regression estimates a continuous value
- Classification predicts a discrete category

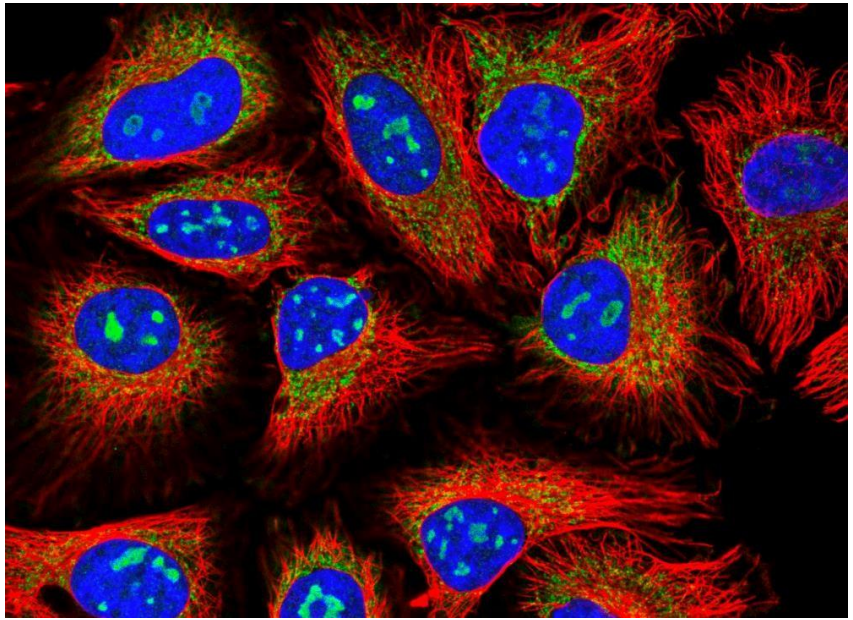
MNIST: classify hand-written digits (10 classes)

ImageNet: classify nature objects (1000 classes)

Figures taken from MNIST Digits Dataset , ImageNet Dataset

Classification Tasks at Kaggle

Classify human protein microscope images into 28 categories



0. Nucleoplasm
1. Nuclear membrane
2. Nucleoli
3. Nucleoli fibrillar
4. Nuclear speckles
5. Nuclear bodies
6. Endoplasmic reticu
7. Golgi apparatus
8. Peroxisomes
9. Endosomes
10. Lysosomes
11. Intermediate fila
12. Actin filaments
13. Focal adhesion si
14. Microtubules
15. Microtubule ends
16. Cytokinetic bridg

Figure taken from Human Protein Atlas Image Classification

Classification Tasks at Kaggle

Classify malware into 9 categories

Figure taken from [Malware Classification]

Classification Tasks at Kaggle

Classify toxic Wikipedia comments into 7 categories

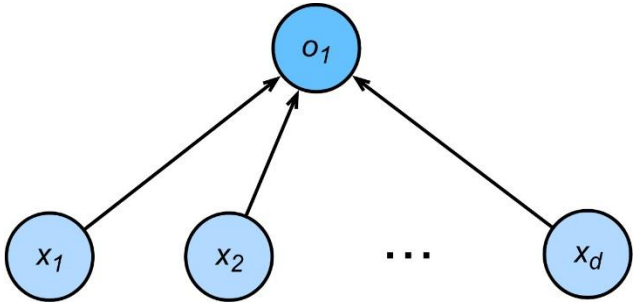
comment_text	toxic	severe_toxic	obs
Explanation\nWhy the edits made under my usern...	0	0	0
D'aww! He matches this background colour I'm s...	0	0	0
Hey man, I'm really not trying to edit war. It...	0	0	0
"\nMore\nI can't make any real suggestions on ...	0	0	0
You, sir, are my hero. Any chance you remember...	0	0	0

Figure taken from [Jigsaw Toxic Comment Classification Challenge]

Multi-class Classification

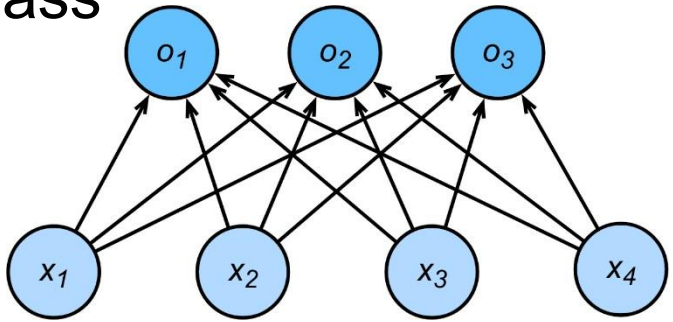
Regression

- Single output value $o_1 \in \mathbb{R}$.
- Output is a real value.
- Loss is defined as difference of $y - \hat{y}$



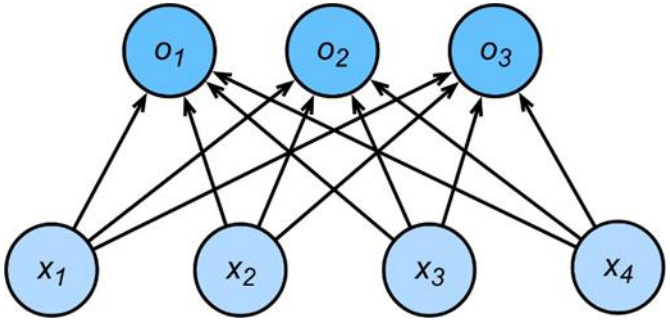
Classification

- Multiple outputs
- Output represents probability i.e. $o_1, o_2, o_3 \in \mathbb{R}$.
- Score represents amount of confidence in predicting a class



Figures taken from Linear Regression , SoftMax Regression

Network Architecture



$$o_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1$$
$$o_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2$$
$$o_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3$$
$$\boldsymbol{o} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

Figure taken from SoftMax Regression

One-hot Encoding

Given $\boldsymbol{y} = [y_1, y_2, y_3, \cdots, y_n]^{\mathrm{T}}$ where n is the number of classes,

$$y_i = \begin{cases} 1, & if\ i = class \\ 0, & otherwise \end{cases}$$

One-hot Encoding - Example

Given $\boldsymbol{y} = [Cat, Dog, Cat, Frog]^{\mathrm{T}}$, the result after one-hot encoding is as follows:

$$\boldsymbol{Y} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

SoftMax

- Model output can be interpreted as probabilities.
- Optimize the parameters to generate probabilities that maximize the probability of the observed data.
- $\hat{\boldsymbol{y}} = softmax(\boldsymbol{o})$
- $\hat{y_i} = \frac{\exp(o_i)}{\sum_{j=1}^{k} \exp(o_j)}$ where k is the number of classes.
- $\hat{y_1} + \cdots + \hat{y_k} = 1$

Vectorized Approach

$$\boldsymbol{X} \in \mathbb{R}^{n \times d}$$
$$\boldsymbol{W} \in \mathbb{R}^{d \times q}$$
$$\boldsymbol{b} \in \mathbb{R}^{1 \times q}$$

Where n is the number of examples, d is the number of features and q is the number of classes.

$$\boldsymbol{O} = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}$$
$$\boldsymbol{Y} = softmax(\boldsymbol{O})$$

SoftMax - Example

Input Image

Model

Logit (O)

3.2
1.3
0.2
0.8

Softmax

$$\hat{y}_i = \frac{\exp(o_i)}{\sum_{j=1}^{k} \exp(o_j)}$$

Probabilities

0.78
0.12
0.04
0.07

Label

Classes
Dog
Cat
Bear
Tiger

Figure taken from Web

Log-Likelihood

- The dataset contains n instances represented by $(\boldsymbol{X}, \boldsymbol{Y})$, where $\boldsymbol{x}^{(i)}$ represents the i^{th} instance and $\boldsymbol{y}^{(i)}$ represents the one-hot encoded label vector.

$$P\ (\boldsymbol{Y}|\boldsymbol{X}) = \prod_{i=1}^{n} P\left(\boldsymbol{y}^{(i)} \middle|\ \boldsymbol{x}^{(i)}\right)$$

- Using Maximum Likelihood (MLE),

$$-\log P\ (\boldsymbol{Y}|\boldsymbol{X}) = \sum_{i=1}^{n} -\log P\left(\boldsymbol{y}^{(i)} \middle|\ \boldsymbol{x}^{(i)}\right) = \sum_{i=1}^{n} -\log P\left(\boldsymbol{y}^{(i)} \middle|\ \boldsymbol{x}^{(i)}\right)$$

Loss Function

- $P\left(\boldsymbol{y}^{(i)}\middle|\boldsymbol{x}^{(i)}\right) = \prod_{j=1}^{q}\left[P\left(y_j^{(i)}\middle|\boldsymbol{x}^{(i)}\right)\right]^{y_j^{(i)}}$
- $P\left(\boldsymbol{y}^{(i)}\middle|\boldsymbol{x}^{(i)}\right) = \prod_{j=1}^{q}\left[\hat{y}_j^{(i)}\right]^{y_j^{(i)}}$
- $-\log P\left(\boldsymbol{y}^{(i)}\middle|\boldsymbol{x}^{(i)}\right) = -\sum_{i=1}^{q} y_j^{(i)}\log\left(\hat{y}_j^{(i)}\right)$
- $-\log P\left(\boldsymbol{y}^{(i)}\middle|\boldsymbol{x}^{(i)}\right) = l\left(\boldsymbol{y}^{(\boldsymbol{i})}, \hat{\boldsymbol{y}}^{(j)}\right)$

Cross Entropy Loss - Example

Given $\hat{\boldsymbol{y}}^{(i)} = \begin{bmatrix} 0.78 \\ 0.12 \\ 0.04 \\ 0.07 \end{bmatrix}$ and $\boldsymbol{y}^{(i)} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, compute $l\left(\hat{\boldsymbol{y}}^{(i)}, \boldsymbol{y}^{(i)}\right)$

- $l\left(\boldsymbol{y}^{(\boldsymbol{i})}, \hat{\boldsymbol{y}}^{(j)}\right) = -\sum_{i=1}^{q} y_j^{(i)} \log\left(\hat{y}_j^{(i)}\right)$
- $l\left(\boldsymbol{y}^{(\boldsymbol{i})}, \hat{\boldsymbol{y}}^{(j)}\right) = -\sum_{i=1}^{4} y_j^{(i)} \log\left(\hat{y}_j^{(i)}\right)$
- $l\left(\boldsymbol{y}^{(\boldsymbol{i})}, \hat{\boldsymbol{y}}^{(j)}\right) = -[\log 0.78 + 0 + 0 + 0]$
- $l\left(\boldsymbol{y}^{(\boldsymbol{i})}, \hat{\boldsymbol{y}}^{(j)}\right) = -\ \log 0.78 = 0.108$

SoftMax and Cross-Entropy Loss

- $l(\boldsymbol{y}, \hat{\boldsymbol{y}}) = -\sum_{j=1}^{q} y_j \log \frac{\exp(o_j)}{\sum_{k=1}^{q} \exp(o_k)}$
- Using log properties we can simply the above expression. The simplified expression is as follows:
- $l(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \sum_{j=1}^{q} y_j \log \sum_{k=1}^{q} \exp(o_k) - \sum_{j=1}^{q} y_j \, o_j$
- Using the fact that at least one $y_j = 1$, we can remove the sum in the first term.
- $l(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \log \sum_{k=1}^{q} \exp(o_k) - \sum_{j=1}^{q} y_j \, o_j$
- Now we take derivative w.r.t logit o_j. Hint [Chain Rule]
- $\frac{\partial l(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial o_j} = \frac{\exp(o_j)}{\log \sum_{k=1}^{q} \exp(o_k)} - y_j = softmax(\boldsymbol{o})_j - y_j$

References

- https://d2l.ai/chapter_linear-classification/index.html